DBMS & SQL — Interview Questions

Database concepts and SQL queries asked in fresher technical interviews. Queries use two tables: `employees(emp_id, name, salary, dept_id, manager_id)` and `departments(dept_id, dept_name)`.

Answer like this in the interview

Define in one line → **explain** with one real-world example → add **one detail** (a difference, a use case or a limitation). Stop at 30–45 seconds and let the interviewer ask the follow-up.

Questions marked ★ **MOST ASKED** come up in almost every fresher technical round — learn those first.

Top questions to learn first

DBMS vs RDBMS · Types of keys · Normalization (1NF–BCNF) · ACID properties · Types of joins · DELETE vs TRUNCATE vs DROP · WHERE vs HAVING · Second highest salary · Primary vs unique key · SQL vs NoSQL

1 · Basics

1 What is a DBMS? How is an RDBMS different? ★ MOST ASKED

A **DBMS** is software that stores, retrieves and manages data (e.g. MongoDB, older file-based systems). An **RDBMS** stores data in **tables (relations)** linked by **keys**, enforces integrity constraints and uses **SQL** — e.g. MySQL, PostgreSQL, Oracle, SQL Server.

Advantages over plain files: less redundancy, consistency, security, concurrent access, backup and recovery.

2 What are a table, tuple, attribute, schema and instance?

A **table (relation)** holds rows and columns. A **tuple** is a row (record); an **attribute** is a column (field). The **schema** is the structure (table names, columns, types); an **instance** is the data stored at one moment.

3 What is an ER model?

An **Entity–Relationship** diagram used to design a database: **entities** (Student, Course) as rectangles, **attributes** as ovals, **relationships** (Enrols) as diamonds, with cardinality 1:1, 1:N or M:N. An M:N relationship becomes a separate junction table, e.g. `enrolments(student_id, course_id)`.

SQL command groups

Group	Purpose	Commands
DDL	Define structure	CREATE, ALTER, DROP, TRUNCATE, RENAME
DML	Change data	INSERT, UPDATE, DELETE
DQL	Read data	SELECT
DCL	Permissions	GRANT, REVOKE
TCL	Transactions	COMMIT, ROLLBACK, SAVEPOINT

2 · Keys & constraints

4 **What are the types of keys?** ★ MOST ASKED

- **Super key** — any set of columns that uniquely identifies a row.
- **Candidate key** — a minimal super key (no extra columns); a table can have several.
- **Primary key** — the candidate key chosen as the main identifier: **unique and NOT NULL**, one per table.
- **Alternate key** — candidate keys not chosen as primary.
- **Foreign key** — a column that refers to the primary key of another table; it links tables and enforces referential integrity.
- **Composite key** — a key made of two or more columns, e.g. (student_id, course_id).

Example: in Student(roll_no, email, name), both roll_no and email are candidate keys; roll_no is primary, email is alternate.

Primary key vs unique key ★ MOST ASKED

Point	Primary key	Unique key
NULL values	Not allowed	Allowed (one NULL in SQL Server; many in MySQL/PostgreSQL)
How many per table	Only one	Many
Default index	Clustered (in most databases)	Non-clustered
Purpose	Identify each row	Prevent duplicate values in a column

5 **What constraints can you put on a column?**

`NOT NULL`, `UNIQUE`, `PRIMARY KEY`, `FOREIGN KEY`, `CHECK` (e.g. `CHECK (age >= 18)`) and `DEFAULT`. Foreign keys can use `ON DELETE CASCADE` to delete child rows automatically.

---

3 · Normalization

6 What is normalization? Why do we need it? ★ MOST ASKED

Organising tables to **remove redundancy** and avoid **anomalies**: **insertion** (can't add data without other data), **update** (same value changed in many places), **deletion** (deleting one fact loses another). It works by splitting tables based on **functional dependencies** (A → B means A determines B).

Normal forms ★ MOST ASKED

Form	Rule	Fix when broken
1NF	Every cell holds a **single atomic value**; no repeating groups	Split multi-valued cells into separate rows
2NF	1NF + **no partial dependency** (a non-key column depends on only part of a composite key)	Move those columns to their own table
3NF	2NF + **no transitive dependency** (non-key → non-key)	Move the dependent columns to a new table
BCNF	For every dependency X → Y, **X is a super key**	Decompose further (stricter 3NF)

7 Explain normalization with an example.

Start: `Order(order_id, product_id, product_name, customer_id, customer_city)` with key (order_id, product_id).

- **2NF**: product_name depends only on product_id (partial) → move to `Product(product_id, product_name)`.
- **3NF**: customer_city depends on customer_id, not on the key (transitive) → move to `Customer(customer_id, customer_city)`.

Result: Order, Product and Customer tables — each fact stored once.

8 What is denormalization?

Deliberately adding redundancy (combining tables or storing computed values) to make **reads faster** by avoiding joins — common in reporting and analytics systems. The trade-off is extra storage and harder updates.

4 · Transactions & concurrency

9 What is a transaction? Explain ACID properties. ★ MOST ASKED

A transaction is a group of operations that run as **one unit of work**. Example: transfer ₹500 from A to B = debit A + credit B.

- **Atomicity** — all or nothing; if the credit fails, the debit is rolled back.
- **Consistency** — the database moves from one valid state to another (total money unchanged).
- **Isolation** — concurrent transactions don't see each other's half-done work.
- **Durability** — once committed, changes survive a crash (written to logs/disk).

10 What problems occur when transactions run concurrently?

- **Dirty read** — reading data another transaction has not committed yet.
- **Non-repeatable read** — the same row gives different values when read twice.
- **Phantom read** — re-running a query returns new rows inserted by another transaction.
- **Lost update** — two transactions overwrite each other's change.

Isolation levels (Read Uncommitted → Read Committed → Repeatable Read → Serializable) prevent more of these as they get stricter.

11 What are locks? What is two-phase locking (2PL)?

A **shared (read) lock** lets many transactions read; an **exclusive (write) lock** lets one transaction write. **2PL**: a transaction first acquires all the locks it needs (growing phase), then only releases them (shrinking phase) — this guarantees serializable schedules. Deadlock can still occur.

12 What is an index? Clustered vs non-clustered?

An index is a data structure (usually a **B+ tree**) that makes searches faster, like a book index — at the cost of slower inserts/updates and extra storage.

Clustered index sorts and stores the actual rows in key order — only one per table. **Non-clustered** index is a separate structure with pointers to the rows — many per table.

5 · SQL concepts

DELETE vs TRUNCATE vs DROP ★ MOST ASKED

Point	DELETE	TRUNCATE	DROP
Type	DML	DDL	DDL
Removes	Selected rows (WHERE allowed)	All rows	Whole table + structure
Rollback	Yes (inside a transaction)	Usually no (yes in PostgreSQL / SQL Server in a transaction)	Usually no
Speed	Slower — logs each row	Fast	Fast
Triggers fire	Yes	No	No

WHERE vs HAVING ★ MOST ASKED

Point	WHERE	HAVING
Filters	Rows, before grouping	Groups, after GROUP BY
Aggregates allowed	No (`WHERE COUNT(*) > 5` is an error)	Yes (`HAVING COUNT(*) > 5`)
Order in the query	Before GROUP BY	After GROUP BY

Types of joins ★ MOST ASKED

Join	Returns
INNER JOIN	Only rows that match in both tables
LEFT JOIN	All rows of the left table + matches from the right (NULL where no match)
RIGHT JOIN	All rows of the right table + matches from the left
FULL OUTER JOIN	All rows of both tables, matched where possible (not in MySQL — use UNION of LEFT and RIGHT)
CROSS JOIN	Every combination — m × n rows (Cartesian product)
SELF JOIN	A table joined with itself, e.g. employee and their manager

13 **What is the order of execution of a SELECT query?**

`FROM` → `JOIN` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT`. This is why a column alias defined in SELECT cannot be used in WHERE.

14 What are a view, a stored procedure and a trigger?

- **View** — a saved SELECT query that acts as a virtual table; used for security and simplicity.
- **Stored procedure** — a named set of SQL statements stored in the database and called with parameters.
- **Trigger** — SQL that runs **automatically** on INSERT, UPDATE or DELETE, e.g. writing an audit log.

15 What are aggregate functions? How does NULL behave?

`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`. They **ignore NULLs**, except `COUNT(*)`, which counts all rows. NULL means unknown: compare with `IS NULL` / `IS NOT NULL`, never `= NULL`.

16 What are UNION and UNION ALL?

Both stack the results of two SELECTs with the same number and type of columns. **UNION removes duplicates** (slower); **UNION ALL keeps them** (faster).

17 What is SQL injection and how do you prevent it?

An attack where user input is inserted into a query, e.g. entering `' OR '1'='1` in a login box. Prevent it with **parameterised queries / prepared statements**, input validation and least-privilege database users — never build SQL by string concatenation.

6 · Queries to write live

18 Find the second highest salary. ★ MOST ASKED

Subquery version works everywhere; for the Nth highest use `DENSE_RANK()` so ties are handled.

```
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- Nth highest (here N = 3)
SELECT DISTINCT salary
FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
      FROM employees) t
WHERE rnk = 3;
```

19 Employees per department, and departments with more than 5 employees. ★ MOST ASKED

Use GROUP BY with COUNT; filter groups with HAVING.

```
SELECT dept_id, COUNT(*) AS emp_count
FROM employees
GROUP BY dept_id
HAVING COUNT(*) > 5;
```

20 Show each employee with their department name, and employees with no department.

INNER JOIN for matches; LEFT JOIN + IS NULL for rows without a match.

```
SELECT e.name, d.dept_name
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id;

SELECT e.name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id
WHERE d.dept_id IS NULL;
```

21 Highest salary in each department, and employees earning above the average.

```
SELECT dept_id, MAX(salary) AS max_salary
FROM employees
GROUP BY dept_id;

SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

22 Find duplicate names and show each employee with their manager (self join).

```
SELECT name, COUNT(*) AS times
FROM employees
GROUP BY name
HAVING COUNT(*) > 1;

SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

23 Names starting with ‘A’, top 3 earners, and salaries between two values.

`%` matches any number of characters, _ exactly one. `LIMIT` is MySQL/PostgreSQL; SQL Server uses `TOP 3`.

```
SELECT * FROM employees WHERE name LIKE 'A%';

SELECT name, salary FROM employees ORDER BY salary DESC LIMIT 3;

SELECT * FROM employees WHERE salary BETWEEN 30000 AND 60000;
```

7 · Beyond relational

SQL vs NoSQL

Point	SQL (MySQL, PostgreSQL)	NoSQL (MongoDB, Cassandra, Redis)
Data model	Tables with a fixed schema	Documents, key-value, column or graph; flexible schema
Relationships	Joins and foreign keys	Usually embedded data, few joins
Scaling	Mostly vertical (bigger server)	Horizontal (more servers, sharding)
Consistency	Strong, ACID	Often eventual consistency (BASE); MongoDB supports transactions
Good for	Banking, orders, structured data	Large-scale, fast-changing or unstructured data

If your project uses a database

Be ready to draw your tables, name the primary and foreign keys, explain one relationship (1:N or M:N) and write a query on your own schema. Interviewers often ask “which normal form is your table in?”

Rapid-fire one-liners

Question	One-line answer
Can a table have two primary keys?	No — one primary key, but it can be composite (several columns).
Can a foreign key be NULL?	Yes, unless declared NOT NULL.
COUNT(*) vs COUNT(column)?	COUNT(*) counts all rows; COUNT(column) skips NULLs.
What is a schema?	The logical structure of the database — tables, columns, types, constraints.
What is a cursor?	A pointer used to process query results row by row.
CHAR vs VARCHAR?	CHAR is fixed length (padded); VARCHAR stores only the actual length.
What is referential integrity?	A foreign key value must exist in the referenced table (or be NULL).
What is a subquery vs correlated subquery?	A correlated subquery uses a column of the outer query and runs once per outer row.
What is data independence?	Changing storage (physical) or schema (logical) without changing applications.
What is RANK vs DENSE_RANK?	On ties RANK skips numbers (1,1,3); DENSE_RANK does not (1,1,2).

Common mistakes interviewers catch

- Writing `WHERE COUNT(*) > 5` — aggregates need HAVING.
- Using `= NULL` instead of `IS NULL`.
- Saying a unique key allows no NULLs — it does allow NULL.
- Saying TRUNCATE can never be rolled back — it depends on the database.
- Mixing up 2NF (partial dependency) and 3NF (transitive dependency).